



Методы оценки синтезированного программного кода и управления его
генерацией с применением языковых моделей

Федрушков Дмитрий Витальевич

Языковые модели пишут всё большую долю производственного кода, но проверять этот код по-прежнему нечем.

Ни один из существующих подходов не даёт комплексной оценки сгенерированного кода:

- Методы опираются на сравнение фрагментов кода, при этом никаким образом или очень косвенно учитывают семантическое значение запроса;
- Существуют методы оценивания на основе тестирования, однако автоматизация данного процесса сталкивается с теми же ограничениями, что и генерация программного кода;
- В оценку кода, генерируемого ИИ агентами, часто не включаются такие параметры, как сложность и безопасность полученного фрагмента;
- Референсное «золотое» решение почти никогда не существует;
- Оценка с помощью ИИ агентов неустойчива и сильно зависит от инструкций.

Зачем оценка, если агенты уже пишут рабочий код



Генерация на сегодняшний день уже не является проблемой, но открытым остаются вопросы, как это верифицировать и как поддерживать:

- OpenAI: бета-продукт около 1 млн строк (код, тесты, CI, документация) за пять месяцев — без единой строки, написанной человеком; pull request'ы ведут сами агенты;
- главной работой инженеров стало построение контуров обратной связи: тестирование, валидация, ревью, обработка обратной связи, восстановление (OpenAI, Harness engineering, 02.2026);

Объём работы ИИ агентов не проверяется вручную:

- в мае 2026 более 70 % пользователей Codex ставили задачи дольше часа работы человека; к июню верхний перцентиль — свыше 60 часов агентных ходов в день на параллельных агентах (OpenAI, 06.2026);

На рассмотрение человека делегируется ровно то, что человек в состоянии проверить:

- ИИ участвует примерно в 60 % работы инженера, полностью делегируется лишь 0–20 % задач — отдают «легко проверяемое на корректность» и малорисковое (Anthropic, 2026).

Узкое место сместилось: не «сгенерировать код», а автоматически оценить результат и решить, что делать дальше

Почему тестов для этого недостаточно

Агенты пишут тесты сами, однако это не даёт надёжного сигнала качества:

- частота написания тестов не различает решённые и нерешённые задачи, объём сгенерированных тестов не влияет на результат; вместо assert-проверок — print-наблюдения (SWE-bench Verified, arXiv:2602.07900, 02.2026);

Тестовое покрытие как единственный инструмент оценки само по себе не является надёжным критерием:

- аудит OpenAI: около 30 % задач SWE-Bench Pro дефектны — тесты подводят в обе стороны: слишком строгие отбраковывают корректные решения, недостаточно покрывающие пропускают неполные исправления (07.2026);

Оценивание — самостоятельная нерешённая задача:

- агенты-ревьюеры решают около 40 % задач бенчмарка с-CRAB (Claude Code 32 %, Devin 25 %, Codex 20 %); на уже исправленных задачах агенты воздерживаются от лишней правки лишь в 36–68 % случаев (ETH Zürich, 03.2026).

Безреференсная оценка не заменяет тесты — она даёт независимый сигнал там, где полного оракула нет, и пригодна для управления генерацией



Нерешённая задача: оценивание качества сгенерированного кода без эталона (безреференсное оценивание) и управление генерацией по этой оценке

Существующие решения и их недостатки

Подходы к оценке сгенерированного кода и их ограничения:

Подход	Достоинство	Недостаток
Запуск тестов (pass@k)	прямая проверка корректности	нужны готовые тесты и изолированная среда — в реальной разработке не всегда доступны
Сходство с эталоном (BLEU, ROUGE, CodeBLEU, BERTScore)	просто, без запуска кода	требуют эталон; корреляция с корректностью ≈ 0 ($ r < 0,14$)
Языковая модель-оценщик (LLM-as-Judge)	без эталона, гибко	неустойчива к формулировке инструкции; иногда не видит локальных структурных дефектов

Нерешённая задача: устойчивая многокритериальная безреференсная оценка, пригодная для управления генерацией

1. Комплементарность оценочных сигналов. Семантическая оценка языковой моделью-судьёй (J) и вероятностные характеристики модели (A) улавливают разные классы дефектов: судья хорошо ранжирует решения глобально, но не видит локальных структурных ошибок, вероятностный сигнал — наоборот. Поэтому по отдельности ни того, ни другого недостаточно.
2. Многокритериальное безреференсное оценивание. Формально проверяемые свойства (синтаксическая корректность, безопасность, вычислительная и структурная сложность) и контекстные сигналы (J, A) сводятся в одну функцию качества $S(x, c)$ с мультипликативной агрегацией и пороговой отбраковкой критических дефектов.
3. Управление генерацией по оценке. Функция S используется как управляющий сигнал: переранжирование кандидатов и итеративная регенерация с диагностической обратной связью — без дообучения языковой модели.

Проблема сигналов, которые даёт ИИ при оценке



Прототип: оценивание языковой моделью-оценщиком — LLM-as-Judge [Zheng et al., 2023].

- Достоинство: оценка без эталона, по смыслу задачи.
- Недостаток: один сигнал — улавливает глобальную семантику, но пропускает локальные структурные дефекты; вердикт не меняется при одиночной ошибке в коде.

Проблема: один семантический сигнал не даёт надёжной оценки качества.

Модель комплементарности сигналов корректности программного кода

$J(T, x)$ — LLM-as-Judge

Эксплицитная семантическая оценка

Свойства:

- Высокая способность к глобальному ранжированию
- Ограниченная локальная диагностика
- Зависимость от инструкций

▲ Ранжирование
▼ Диагностика

$\text{Rank}(J(x_1)) > \text{Rank}(J(x_2))$
⇒ корректность $x_1 >$ корректность x_2

$A(T, x)$ — Log-probability delta-based anomaly score

Оценка относительной неожиданности токенов

Свойства:

- Высокая чувствительность к локальным аномалиям
- Ограниченная способность к глобальному ранжированию
- Независимость от инструкций

▼ Ранжирование
▲ Диагностика

$\Delta_i = \log P(t_i | c_i) - \text{mean}(\text{top-k log } P(t | c_i))$
⇒ наличие токенов с $\Delta_i \ll 0$ указывает на локальные аномалии

Формализация комплементарности

Свойство 1:

$\exists x_1, x_2 \in X(T): A(T, x_1) \approx A(T, x_2), J(T, x_1) \neq J(T, x_2)$

Свойство 1: Стилистически близкий, но семантически различный код получает близкие значения вероятностных оценок, но различные значения оценок Judge

Свойство 2:

$\exists x_1, x_2 \in X(T): J(T, x_1) \approx J(T, x_2), A(T, x_1) \neq A(T, x_2)$

Свойство 2: Семантически близкий код с нетипичным выбором операций или конструкций получает близкие значения оценок Judge, но различные значения вероятностных оценок

Следствие: $J(T, x)$ и $A(T, x)$ — слабо коррелированные оценки, определяющие разные классы ошибок
⇒ оптимальная оценка требует их комбинации ($\alpha, \beta \neq 0$)

Отличие от известных подходов

Прототип (LLM-as-Judge)	Предложенная модель комплементарности
Один оценочный сигнал	Два взаимодополняющих сигнала: семантический J и вероятностный A
Только глобальная семантика	Глобальная семантика + локальная диагностика дефектов
Взаимодополняемость сигналов не исследована	Комплементарность доказана количественно (инверсия ROC-AUC)

С целью повышения полноты и устойчивости безреференсной оценки совместно используются два сигнала с доказанной комплементарностью

Результаты экспериментов с оценивающими сигналами

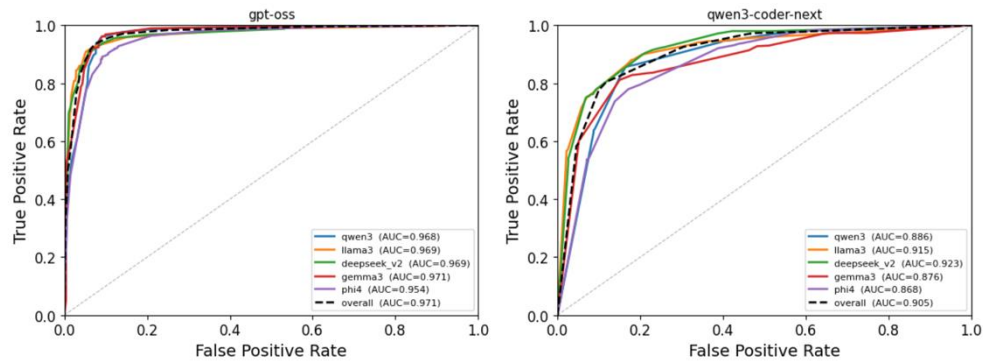


Рис 1. Языковые модели как оценщики (оценивание сгенерированного кода)

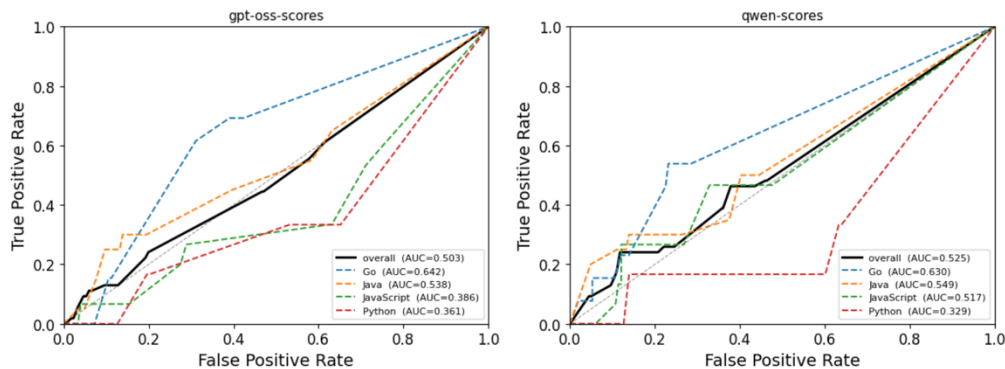
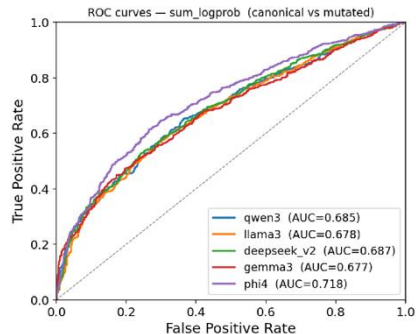
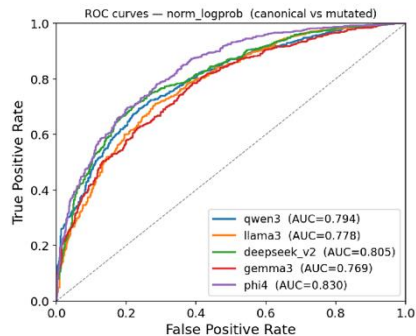


Рис 2. Языковые модели как оценщики (оценивание мутированного кода)

Результаты экспериментов с оценивающими сигналами



(a) sum_logprob



(b) norm_logprob

Рис 3. Логарифмы вероятностей решений – способность различать мутированное решение (a – суммарный логарифм вероятности, b – нормализованный логарифм вероятности)

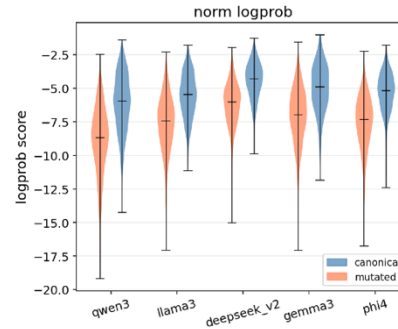


Рис 4. Логарифмы вероятностей решений – распределение нормализованных вероятностей

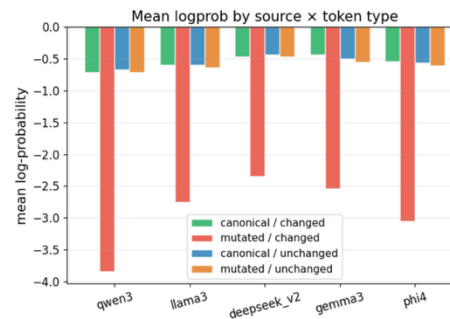


Рис 5. Логарифмы вероятностей токенов решений – сравнение оригинальных и мутированных решений

Примеры



Показатель	Значение
Общее число пар решений	8196
Число пар, удовлетворяющих свойству 1	203
Доля (свойство 1)	2,48%
Число пар, удовлетворяющих свойству 2	1959
Доля (свойство 2)	23,9%

Java/53 (Java) **gpt-oss:0.97** **qwen:0.95** ' +' [0.00] → ' -' [-11.00]

canonical (8 tok):

```
return x + y ; ~ ~ ~ ~ }
```

mutated (8 tok):

```
return x - y ; ~ ~ ~ ~ }
```

JavaScript/45 (JavaScript) **gpt-oss:0.05** **qwen:0.00** ' 2' [-0.12] → ' -1' [-8.69]

canonical (11 tok):

```
return a * b / 2 . 0 ; ~ }
```

mutated (11 tok):

```
return a * b / -1 . 0 ; ~ }
```

A simple program which should return the value of x if n is a prime number and should return the value of y otherwise.

1 - Judge = 0,0000 | Logprob = -0,8152

```
func XORY(n, x, y int) int {
    if n%2 == 0 {
        return y
    }
    else {
        return x
    }
}
```

2 - Judge = 0,6000 | Logprob = -0,8199

```
func XORY(n, x, y int) int {
    for i := 2; i < n; i++ {
        if n%i == 0 {
            return y
        }
    }
    return x
}
```

Свойство 1: $A(x_1) \approx A(x_2)$, но $J(x_1) \neq J(x_2) \rightarrow$
стилистически похожий, но семантически разный код

Given the lengths of the three sides of a triangle. Return True if the three sides form a right-angled triangle, False otherwise. A right-angled triangle is a triangle in which one angle is right angle or 90 degree.

1 - Judge = 0,0700 | Logprob = -1,9375

```
def right_angle_triangle(a, b, c):
    if a*a + b*b == c*c:
        return True
    return False
```

2 - Judge = 0,0500 | Logprob = -1,0027

```
def right_angle_triangle(a, b, c):
    if a**2 + b**2 == c**2:
        return True
    else:
        return False
```

Свойство 2: $J(x_1) \approx J(x_2)$, но $A(x_1) \neq A(x_2) \rightarrow$
семантически похожий код с разной вероятностной типичностью

Рис 1. Примеры дискриминации
аномальных токенов

Инверсия ROC-AUC между корпусами:

- LLM-as-Judge: 0,971 на сгенерированном коде $\rightarrow$ 0,503 на мутированном (уровень случайного угадывания).
- Log-probability: наоборот — попарная точность $> 97\%$ на мутациях; отклонения локализуются на изменённых позициях.

Вывод: сигналы покрывают взаимно непересекающиеся классы дефектов $\rightarrow$ их совместное использование обосновано.

Проблема использования ИИ для оценки



- Прототип: CodeJudge [Tong et al., 2024] — безреференсный оценщик кода на основе LLM. Двухшаговая процедура: анализ кода относительно задачи → классификация несоответствий. Не требует эталона и запуска кода.
- Лучший из известных (SOTA) среди безреференсных оценщиков: Spearman $\rho \approx 0,61$ на HumanEval-X (выше ICE-Score 0,49, CodeBLEU 0,44, BLEU 0,37).
- Недостаток: единственный семантический сигнал, полностью ответственный за всю оценку кода – крайне зависит от инструкций.

Критерии оценивания синтезированного программного кода



Критерий	Инструмент
Корректность	Языковая модель – критик (LLM as a judge)
Аномальность	Вероятностные характеристики языковых моделей
Безопасность	Статистические инструменты анализа
Вычислительная сложность	Эвристическая оценка O-нотации по времени и по памяти
Читаемость	Оценки структурной сложности

Разные типы сигналов обеспечиваются разными специализированными инструментами



$$S(x, c) = J(x, c) \cdot \text{безопасность}(c) \cdot Q(x, c)$$

Q — взвешенное среднее вероятностной оценки A и оценок сложности; критический дефект ($J \leq \tau$) $\rightarrow S = 0$

Отличие от известных подходов

Однокритериальная оценка	Многокритериальная функция $S(x, c)$
Один критерий	Формальные (безопасность / сложность / читаемость) + контекстные (J, A)
Простое или одиночное объединение	Мультипликативная агрегация с пороговой отбраковкой
Нет защиты от критических дефектов	Критический дефект обнуляет итоговую оценку

С целью повышения устойчивости, воспроизводимости и интерпретируемости оценки сигналы объединяются в единую многокритериальную функцию

Экспериментальные результаты

Метрика	Среднее (pass)	Среднее (fail)	Дельта Клиффа	Эффект	p-value (MWU)	Корреляция Спирмена ρ	p-value (ρ)	Значимость
J(T, x) (Judge)	0,9484	0,8764	0,2778	небольшой	8,46e-21	0,2563	3,99e-21	✓
A(T, x) (Logprob bucket)	0,8376	0,7978	0,0404	незначительный	1,15e-01	0,0332	2,29e-01	✗
J(T, x) · A(T, x)	0,7944	0,6985	0,2802	небольшой	3,77e-17	0,2303	2,98e-17	✓
S(T, x)	0,7837	0,7033	0,2852	небольшой	1,07e-17	0,2344	7,92e-18	✓
Safety	0,9928	0,9978	-0,0048	незначительный	8,31e-01	-0,0264	3,39e-01	✗
Compute Complexity	0,8277	0,8221	0,0105	незначительный	3,74e-01	0,0089	7,48e-01	✗
Structural Complexity	0,8113	0,7987	0,0370	незначительный	1,35e-01	0,0304	2,71e-01	✗

Результаты ИИ оценщиков



Оценщик	ROC-AUC · судья Gemma3	ROC-AUC · судья Phi-4
CodeJudge (прототип, лучший из известных)	0,606	0,496
J: эксплицитный LLM-судья	0,624	0,670
A: вероятностный сигнал log-prob	0,765	0,730
J × A: связка судьи и log-prob	0,770	0,747

HumanEval-X, n = 656 на судью, кросс-оценка (модель-оценщик ≠ модель-генератор). A и связка J × A используют вероятностный сигнал ($\sum \log p$), недоступный CodeJudge.

Вывод: один семантический сигнал (CodeJudge) недостаточен — на судье Phi-4 он ≈ случаен (AUC 0,50); связка «судья + вероятностный сигнал» доминирует (AUC 0,75–0,77). Формальные свойства CodeJudge не оценивает.

Выводы по свойствам внешних оценщиков

Функция $S(x, c)$ статистически значимо различает корректные и некорректные решения:

- Cliff $\Delta = 0,285$; $p \approx 1,1 \cdot 10^{-17}$
- Наибольший размер эффекта среди всех рассмотренных вариантов оценки

Классические метрики сравнения с эталоном (BLEU, ROUGE, CodeBLEU, BERTScore) — корреляция с корректностью ≈ 0 . CodeJudge оценка статистически реже различает корректные и некорректные решения.

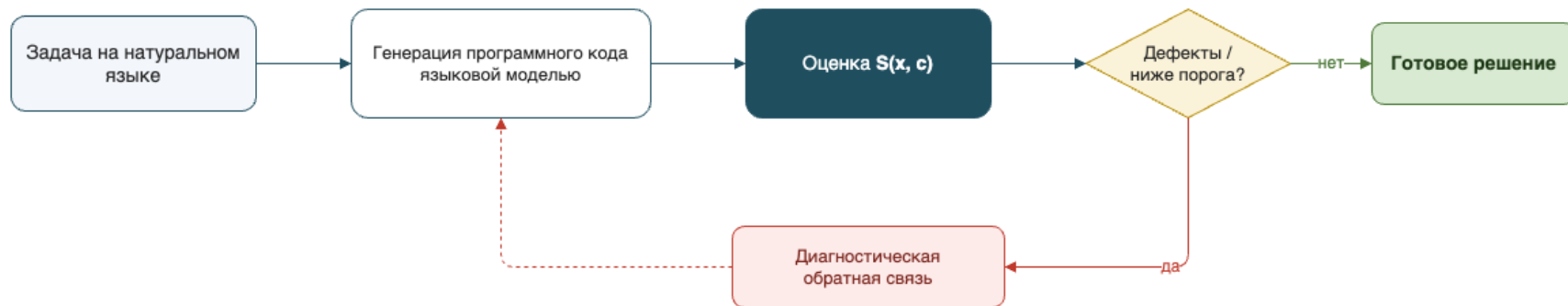
Проблема генерации без управляющего сигнала



Прототип: генерация в стандартном режиме без учета компонентов, если не указано в инструкциях. ИИ агент замкнут сам на себя

- Недостаток: оценивание выполняется после генерации и на сам процесс не влияет; нет обратной связи; синтаксические галлюцинации не отсекаются.

Проблема: как получить более корректный результат с точки зрения нескольких критериев без дообучения (изменения весов) модели и ручного инструктирования?



Управляющее правило: $c^* = \operatorname{argmax}_i S(x, c_i)$

переранжирование кандидатов и регенерация с обратной связью, пока $S(x, c) \leq \tau$ (порог); не более T итераций

Отличие от известных подходов

Стандартная генерация	Управление генерацией
Оценка после генерации – семантическая, самим агентом	Оценка управляет генерацией по нескольким критериям, не только по семантике
Одиночный проход / Цикл генерации – код -> тесты -> код	Переранжирование (при нескольких вариантах) + регенерация с обратной связью

С целью повышения доли корректных решений без изменения весов модели результаты оценивания используются как управляющий сигнал генерации

Экспериментальные результаты



Язык	Генератор	Judge/Logprob модель	Pass (baseline)	Pass (с фидбеком)	Δ (delta)
Java	Gemma3-12B	Phi-4/Qwen3-14B	0,5915	0,6220	+0,0305
Java	Gemma3-12B	Phi-4/Phi-4	0,5915	0,6220	+0,0305
Java	Phi-4	Gemma3-12B/Qwen3-14B	0,6524	0,6341	-0,0183
Java	Phi-4	Gemma3-12B/Gemma3-12B	0,6524	0,6341	-0,0183
Python	Gemma3-12B	Phi-4/Qwen3-14B	0,6890	0,7012	+0,0122
Python	Gemma3-12B	Phi-4/Phi-4	0,6890	0,6890	+0,0000
Python	Phi-4	Gemma3-12b/Qwen3-14B	0,6463	0,6829	+0,0366
Python	Phi-4	Gemma3-12B/Gemma3-12B	0,6463	0,6768	+0,0305

Генерация с комбинированной обратной связью

Эффект от компонентов языковых моделей

Язык	Генератор	Judge модель	Pass (baseline)	Pass (с фидбеком)	Δ (delta)
Java	Gemma3-12B	Phi-4	0,5915	0,6118	+0,0203
Java	Phi-4	Gemma3-12b	0,6524	0,6423	-0,0102
Python	Gemma3-12B	Phi-4	0,6890	0,6951	+0,0061
Python	Phi-4	Gemma3-12b	0,6463	0,6707	+0,0244

Генерация с обратной связью от Judge

Язык	Генератор	Logprob модель	Pass (baseline)	Pass (с фидбеком)	Δ (delta)
Java	Gemma3-12B	Qwen3-14B	0,5915	0,6220	+0,0305
Java	Gemma3-12B	Phi-4	0,5915	0,6128	+0,0213
Java	Phi-4	Qwen3-14B	0,6524	0,6341	-0,0183
Java	Phi-4	Gemma3-12b	0,6524	0,6341	-0,0183
Python	Gemma3-12B	Qwen3-14B	0,6890	0,6921	+0,0030
Python	Gemma3-12B	Phi-4	0,6890	0,6921	+0,0030
Python	Phi-4	Qwen3-14B	0,6463	0,6799	+0,0335
Python	Phi-4	Gemma3-12B	0,6463	0,6677	+0,0213

Генерация с обратной связью от Logprob

Эффект на формальных компонентах

Полная система (перегенерация по многокритериальному сигналу S) улучшает не только pass@1, но и формальные свойства сгенерированного кода.

Формальная компонента	Δ среднее	95% ДИ	улучшилось
Безопасность	+0,005	[+0,002; +0,008]	1%
Вычислительная сложность	+0,006	[+0,004; +0,008]	6%
Структурная сложность	+0,025	[+0,021; +0,029]	20%

Дополняет прирост pass@1 (+3,66 п.п.): управляющий сигнал S улучшает и функциональную корректность, и формальные свойства кода — без изменения весов модели.

Часть 3. Пример обратной связи



Prompt:

```
import java.util.*;
import java.lang.*;

class Solution {
    /**
     Given two positive integers a and b,
     return the even digits between a
     and b, in ascending order.

     For example:
     generateIntegers(2, 8) => [2, 4, 6, 8]
     generateIntegers(8, 2) => [2, 4, 6, 8]
     generateIntegers(10, 14) => []
     */
    public List<Integer> generateIntegers(int
a, int b) {
```

Model answer:

```
List<Integer> evenNumbers = new ArrayList<>();
    if (a > b) {
        int temp = a;
        a = b;
        b = temp;
    }

    for (int i = a; i <= b; i++) {
        if (i % 2 == 0) {
            evenNumbers.add(i);
            Collections.sort(evenNumbers);
        }
    }

    return evenNumbers;
}
```

Feedback:

=== Automated metrics (score: 0,804) ===
Code analysis results:

Judge: 0,700
Anomaly: 0,924
Safety: 1,000
Compute Complexity: 0,590
Struct. Complexity: 0,721

Overall: 0,8037

Issues:

- LLM Judge rated solution 0,70/1,0 (partially correct). Review the requirements.
- The function collects even digits from every number between a and b, not just those that fall strictly between the two values as individual integers.
- The task implies returning even numbers in the range [a, b], but the solution returns all even digits found within any number in this range

Remarks:

* Unexpected token 'import' at line 1:1 - may indicate a structural issue

Performance / maintainability:

~ High algorithmic complexity $O(n^2)$ (heuristic estimate, may be acceptable depending on input size)

Часть 3. Выводы

Формирование диагностической обратной связи на основе многокритериального оценивания позволяет использовать оценку для повторной генерации программного кода и обеспечивает:

- прирост pass@1 до +3,66 п.п. на исследованных конфигурациях;
- повышение качества достигается без изменения весов языковой модели;
- дополнительно улучшаются формальные свойства программного кода.

Предложенная функция качества выступает управляющим сигналом, обеспечивающим повышение функциональной корректности и формальных характеристик программного кода без дообучения языковой модели.

Выводы

После исследования задачи безреференсного оценивания качества синтезированного программного кода и управления его генерацией, без эталонного решения и без изменения весов языковой модели, основные результаты заключаются в следующем:

- Результат 1 — доказана комплементарность семантического (J) и вероятностного (A) сигналов: инверсия ROC-AUC ($0,97 \rightarrow 0,50$ у судьи на мутациях, $> 97\%$ попарной точности у log-prob) обосновывает их совместное использование;
- Результат 2 — предложена многокритериальная функция $S(x, c)$, объединяющая формальные и контекстные критерии; связка «судья + вероятностный сигнал» превосходит SOTA-прототип CodeJudge (ROC-AUC $0,75-0,77$ против $0,50-0,61$);
- Результат 3 — оценка S используется как управляющий сигнал генерации: прирост pass@1 до +3,66 п.п. и улучшение формальных свойств кода без дообучения модели.

Вместе модель комплементарности, функция качества $S(x, c)$ и метод управления генерацией образуют единый подход к безреференсной оценке кода и её использованию в контуре генерации.



Спасибо за внимание!

Федрушков Дмитрий Витальевич
e-mail: dvfedrush@gmail.com